



Security Audit

Kingdomly (Staking)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Behaviours	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Audit Findings	12
Centralisation	38
Conclusion	39
Our Methodology	40
Disclaimers	42
About Hashlock	43

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Kingdomly team partnered with Hashlock to conduct a security audit of their Staking Contracts including KingdomlyStaking.sol & KingdomlyFeeContract.sol. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed contracts were secure.

Project Context

Kingdomly is a platform that enables users to create and manage NFT collections with ease. It provides tools for launching generative or unique NFTs without requiring technical skills. The platform handles the entire NFT creation process, including smart contracts and metadata, and offers features like customizable mint pages, private sales, and token-gated applications. Kingdomly aims to simplify NFT creation, allowing creators to focus on their art and community engagement.

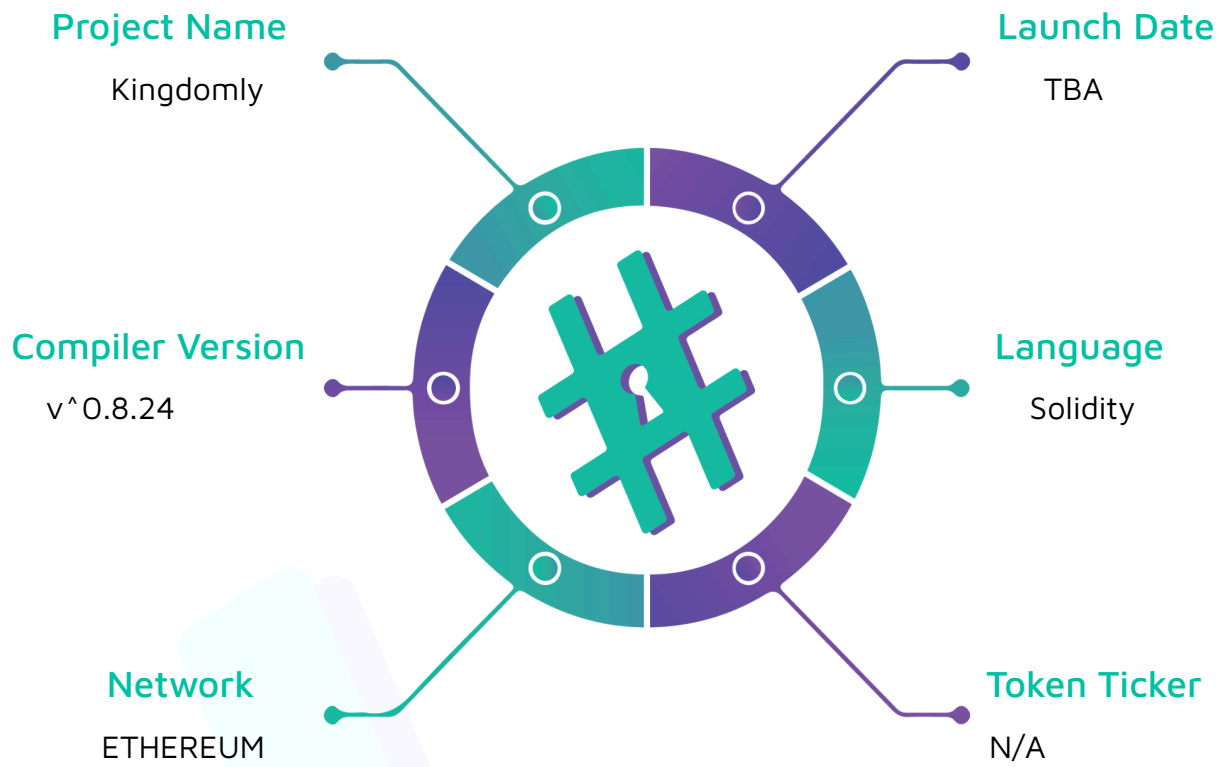
Project Name: Kingdomly

Compiler Version: ^0.8.24

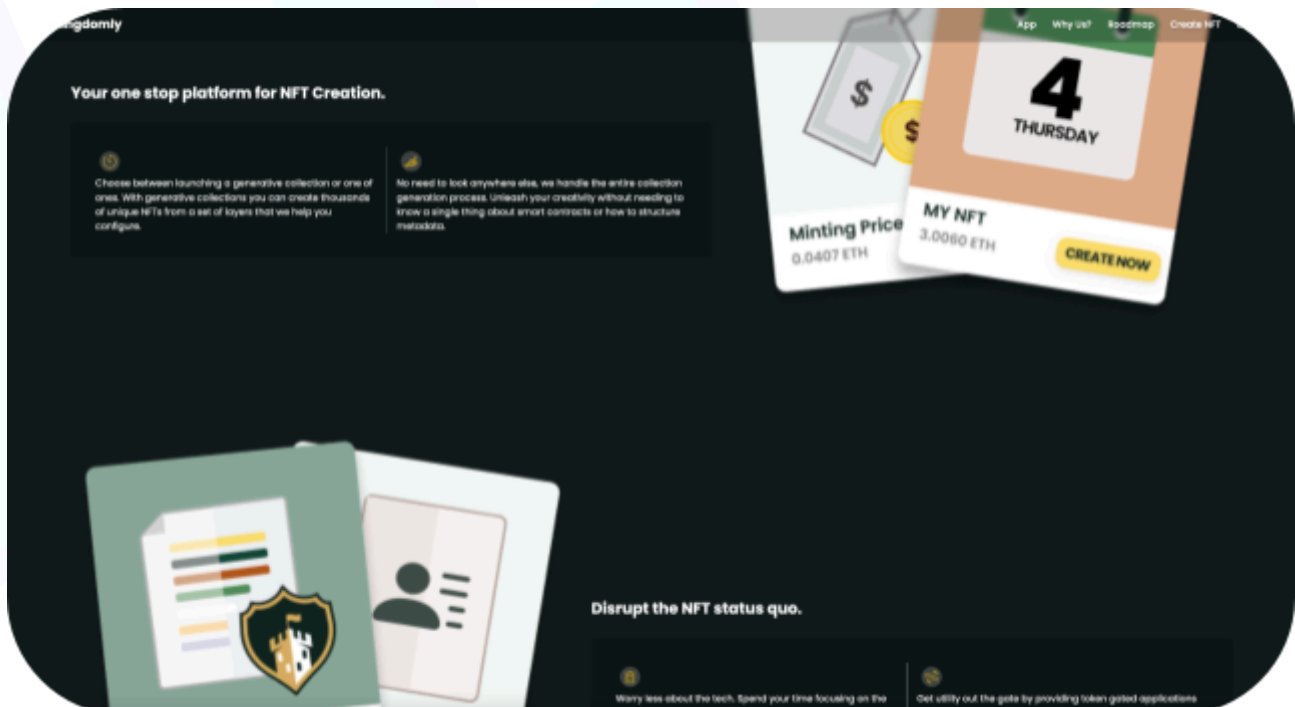
Website: <https://www.kingdomly.info/>

Logo:



Visualised Context:

Project Visuals:



#Hashlock.

Hashlock Pty Ltd

Audit scope

We at Hashlock audited the solidity code within the Kingdomly Project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Kingdomly Staking Smart-Contracts
Platform	Ethereum / Solidity
Audit Date	August, 2024
Contract 1	KingdomlyStaking.sol
Contract 1 MD5 Hash	4d691875cb26a681d30039510b3d975c
Contract 2	KingdomlyFeeContract.sol
Contract 2 MD5 Hash	1bcc2e32e6933698edf9cbb46b660721

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, Pyth Oracle Contracts and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section.

We initially identified some significant vulnerabilities that have since been addressed.

Hashlock found:

5 High-severity vulnerabilities

4 Medium-severity vulnerabilities

5 Low-severity vulnerabilities

1 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
KingdomlyStaking.sol Allows users to: - Stake their NFT - Withdraw their Staked NFT - Claim their RewardTokens Allows admins to: - Set & Deposit RewardToken - Increase the amount of RewardToken in Pool - Withdraw the RewardToken - Start the Redemption Period for users - Start the RedemptionOff-Chain Calculations	Contract achieves this functionality.
KingdomlyFeeContract.sol Allows admins to: - Able to retrieve OneDollarInWei - Can update the Oracle Prices - Add Verified contracts to make calls to this contract - Remove contracts to make calls to this contract	Contract achieves this functionality.

Code Quality

This Audit scope involves the smart contracts of the Kindomly Project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Kindomly project smart contract code in the form of a Github file.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help us understand the overall architecture of the protocol.

Dependencies

Per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry-standard open-source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies

Audit Findings

High

[H-01] KingdomlyStaking#_withdraw - Using the same TokenId in an indefinite Array leads to DOS the contract from further use.

Description

During the user's withdrawal process, the user can provide an indefinite array with the same TokenId in the parameters leading to the DOS of the contract from further use making the protocol useless.

Vulnerability Details

In normal scenarios, Users can stake "single" or "multiple" NFTs by providing them in an array while calling the stake() function. Later we add the number of tokens that the user has provided directly into the `totalStakedTokens` state-variable typically increasing the count.

Later on, while withdrawing the NFTs from the protocol, the contract decreases the `totalStakedTokens` variable by doing adequate checks.

The issue arises when a user typically provides the same 'TokenId' multiple times in an array leading to a decrease in the value of `totalStakedTokens` subsequently leading to DOS the whole contract.

As the decrease of the number of tokens Staked is dependent on `tokenIds.length`, and as the number of checks are insufficient

```

function withdraw(IERC721 _collection,uint256[] calldata _tokenIds) external payable
nonReentrant updateStoredPoints(msg.sender, address(_collection), _tokenIds) {
    ...
    ...
    // set accumulated rewards per share for collateral asset
    for(uint256 i; i < _tokenIds.length;){
        delete stakedAsset[address(_collection)][_tokenIds[i]];

        ...

        ...

        userStakedTokens.pop()

        _collection.safeTransferFrom(address(this), msg.sender, _tokenIds(i));
>     totalStakedTokens -= _tokenIds.length;

        ...

        ...

        emit Withdraw(msg.sender, _tokenIds, address(_collection))

    }
}

```

Proof of Concept

Assume this scenario:

USER A - stakes 1 NFT

USER B - stakes 45 NFTs

USER C - stakes 4 NFTs

Now, the value/count of the `totalStakedTokens` will be 50. Now, USER A has decided to DOS the contract so he's done the following:

- USER A calls the function `withdraw()` with the following parameters:

> IERC721 `_collection`

> uint256[] calldata `tokenIds`

```

uint256[] tokenIds;

tokenIds[0] = NFTOfUSERA;
tokenIds[1] = NFTOfUSERA;
tokenIds[2] = NFTOfUSERA;
tokenIds[3] = NFTOfUSERA;
tokenIds[4] = NFTOfUSERA;
tokenIds[5] = NFTOfUSERA;
tokenIds[6] = NFTOfUSERA;
tokenIds[7] = NFTOfUSERA;
tokenIds[8] = NFTOfUSERA;
tokenIds[9] = NFTOfUSERA;

```

- Upon calling the function, the modifier `updateStoredPoints()` does some checks
 - here the core issue is that even this modifier doesn't check for duplicate values
 - leading to the increase in stored Points – CHECK H-02
- Once reached the for-loop statement, this loops 10 times - as our `tokenIds.length` is 10. And immediately on the next line, it deletes the mappings and resets the value to null.
- And then, it sends back the NFT to the `msg.sender`.

To exploit the `msg.sender` (a contract) writes a fallback function that typically works:

```

function ONIERC721Received() external returns(bytes4){
    NFT.transferFrom(address(this), StakingContract, NFTTokenId);

    Return ONIERC721Received.selector;
}

```

- As the NFT is still in the `StakingContract` at this moment, we can able to run this loop without getting the issue of "Token Doesn't exist" or such.
- Then, once the loop is finished, it makes the state changes to the variable `totalStakedTokens` directly subtracting the `_tokenIds.length`. In our case, it does the following

```
totalStakedTokens -= tokenIds.length

// 50 -= 10

// totalStakedTokens = 40
```

- Now, if USER B comes to withdraw all his NFTs, he won't be able to do it as the totalStakedTokens would revert as it would cause underflow.

Impact

Any user can stake the NFT and can able to DOS the whole contract by simply providing the same staked TokenId in an array typically stopping other users from withdrawing their stakedTokens.

In our example/POC, if USER A has provided the array length of 50 instead of 10, it would indefinitely cause the protocol to brick as no one could able to withdraw their StakedNFTs nor anyone could able to redeem their rewards.

Recommendation

Instead of subtracting the values from user-inputted arrayLength, Do it from the count using tokensStaked mapping which keeps track of all the tokens staked by the user.

Status

Resolved

[H-02] KingdomlyStaking#updateStoredPoints - Users could able to redeem more rewards than usual leading to stealing other user rewards from the protocol.

Description

While the user is calling the `withdraw()` on the KingdomlyStaking contract, two modifiers work alongside the main code: `nonReentrant` - To stop Reentrancy Attacks and `updateStoredPoints` - To calculate the points of the user for their stakedNFTs. But upon giving the same TokenId multiple times in an array, it adds the ability to get more points than usual.

Vulnerability Details

Whenever any user calls the `withdraw()` function with the collection address and array of `TokenIds`, it first calls the `updateStoredPoints` modifier to add up all the points for the user to claim in the future upon activating the redemption period.

```
modifier updateStoredPoints(address _user, address _collection, uint256[] calldata tokenIds){  
    ...  
    for(uint256 i; i < tokenIds.length;){  
        ...  
        uint256 newUserRewards = calculateUserPointsFromStakedTokens(_user,_collection..)   
        storedPoints[_user] += newUserRewards;  
    }  
}
```

However, The issue becomes valid if the user tries to give the same `TokenId` multiple times in an array, and by utilizing the previously mentioned issue(H-01), he could able to complete the transaction without any reverts making the state-changes permanent.

This in return causes the protocol to lose funds when this user tries to redeem the `RewardTokens`, he'll get more tokens than usual.

Proof of Concept

Assume the `KingomdlyStaking` contract is having 1M `USDC` as `RewardTokens`. These were the following values:

```
totalPoolPoints = 1000;  
redeemPoints[USERA] = 300;  
redeemPoints[USERB] = 700;
```


According to the code at `getUserRedeemPointsPercentage()`, the calculation to find the percentage of user's `redeemPoints` respective to total pool points is:

```
_addressPointsPercentage = ( redeemPoint[USER] * 1_000_000 ) / totalPoolPoints
```

At this point, USER A percentage is: $(300 * 1_000_000) / 1000 = 3,00,000$

And USER B percentage is: $(700 * 1_000_000) / 1000 = 7,00,000$

In the `redeemRewards()` function, the `tokenRewardAmount` calculation works like this:

```
tokenRewardAmount = ( depositedRewardAmount * userRewardPercentage ) / 1_000_000
```

FOR USER A: $(10_00_000 * 3,00,000) / 1_000_000 = 300K$ USDC Tokens

FOR USER B: $(10_00_000 * 7,00,000) / 1_000_000 = 700K$ USDC Tokens respectively.

But let's say there's a USER C who did the following:

- USER C Stakes an NFT and keep it for X days.
- USER C calls the `withdraw()` function with array of `tokenIds` with length of 10.
- The modifier `updateStoredPoints()` calls internal function, `calculateUserPointsFromStakedTokens` and let's say it returned 100 as output.
- Now due to the lack of verification of double-checking of same `TokenId`, the points are added to `StoredPoints` mapping and later to the `totalPoolPoints` state variable.
- Upon it, let's say the `storedPoints[USER C]` is 1000 (As we run the loop for 10 times) and our `totalPoolPoints` will be 2000.

Now let's do the calculations:

```
totalPoolPoints = 2000;

redeemPoints[USERA] = 300;

redeemPoints[USERB] = 700;
```

```
redeemPoints[USERC] = 1000;
```

And as per the logic, the UserPercentageOfPool Tokens were:

```
_addressPointsPercentage = ( redeemPoint[USER] * 1_000_000 ) / totalPoolPoints
```

USER C percentage = $(1000 * 1_000_000) / 2000 = 5,00,000$

And when USER C calls the redeem() function, he'll get the RewardTokens worth the amount of tokenRewardAmount:

```
tokenRewardAmount = ( depositedRewardAmount * userRewardPercentage ) / 1_000_000
```

tokenRewardAmount = $(10_00_000 * 5_00_000) / 1_000_000 = 5,00,000$

So technically here, the USER C is claiming 50% of the RewardTokens in the Pool subsequently decreasing the reward for users who've staked for a longer duration.

Impact

By utilizing this issue, any USER can able to stake for a short duration but be able to get more StoredPoints by looping multiple times allowing them to steal the RewardTokens and also hurting other users from claiming their rewards who've staked tokens for a longer time.

Recommendation

It is recommended to check the tokenIds inside the array and not allow the token to exist multiple times inside an array.

Status

Resolved

[H-03] KingdomlyStaking#withdrawRewardToken - Wrong Token Calculations don't allow the Admin to withdraw the RewardTokens and permanently stuck in the contract

Description

Admin can able to withdraw the RewardTokens from the contract whenever there is an issue with the StakingContract or for any other reason, but because of the wrong calculation and parameters, Admin couldn't able to withdraw the RewardTokens making them permanently stuck in the contract.

Vulnerability Details

Let's say the Admin sets the RewardToken as USDC/USDT(or any other weird-Token) and in the future, or present if the RewardToken is having fee-on transfer i.e it deducts some balance while transferring, it creates the issue of DOS for this contract.

Assume Admin deposits 1M USDC/USDT/Fee-On-Transfer tokens and let's say there is a fee of 5 Tokens for every transaction. If the Admin calls the `depositRewardToken()` function by specifying the token address and amount, it works as follows:

```
function depositRewardToken(IERC20 _token,uint256 _amount)external OnlyOwner
rewardRedemptionActive(false){
    ...
    ...
    rewardToken = _token;
> depositedRewardTokenAmount = _amount;
> rewardToken.safeTransferFrom(msg.sender, address(this), _amount);
    emit RewardTokenDeposited(address(this), _amount);
}
```

As we're setting up the state variable 'depositedRewardTokenAmount' directly to the given amount without actually looking up the balanceOf(), it can cause issues while withdrawing.

Now After our example, the value in depositedRewardAmount = 1M Tokens but the actual token.balanceOf(address(this)) = 1M - 5(Fee)

Let's say now, the Admin wants to withdraw the RewardTokens and calls the withdrawRewardToken() function

```
function withdrawRewardToken() external OnlyOwner rewardRedemptionActive(false){
    rewardToken.safeTransfer(msg.sender, depositedRewardAmount);
    emit RewardTokenWithdrawn(address(this), depositedRewardAmount);
}
```

But as this contract currently holds only 99,99,95 Tokens, it reverts and isn't able to succeed regardless of how many times the Admin tries to withdraw.

Along with it, as there is no partial-token amount withdrawal option, it doesn't work either leading to the High severity.

Impact

Due to this issue, if there is any issue with the contract and Admin tries to withdraw the RewardTokens, it doesn't work and the tokens will be permanently stuck in the contract as there is no option to withdraw Tokens either.

Recommendation

Change the function & also add another function to allow partial withdrawal of tokens.

```
function withdrawRewardToken() external OnlyOwner rewardRedemptionActive(false){
    if(address(rewardToken) == address(0)){
        revert InvalidTokenAddress;
    }
    rewardToken.safeTransfer(msg.sender, rewardToken.balanceOf(address(this)));
}
```

```

        emit RewardTokenWithdrawl(address(rewardToken),
rewardToken.balanceOf(address(this)));
    }

```

```

function withdrawPartial(uint256 _amount) external OnlyOwner
rewardRedemptionActive(false){

    if(address(rewardToken) == address(0)){

        revert InvalidTokenAddress;

    }

    rewardToken.safeTransfer(msg.sender, _amount);

    emit RewardTokenWithdrawl(address(rewardToken), _amount);

}

```

Status

Resolved

[H-04] KingdomlyStaking#stake/withdraw - Users won't be able to execute Stake() or withdraw() function upon sending more native tokens than required

Description

In normal conditions, users are required to pay the nominal fee also known as `StakingFee` to the platform for interactions such as `stake()` & `withdraw()` and also exists in the contract `KingdomlyFeeContract.sol/updateOracleAndGetOneDollarInWei` but there is a strict functionality check which always tends to revert if the users try to send more native-tokens as Fee than required.

Vulnerability Details

Staking the NFT or Withdrawing the NFT requires the User to pay some fee to the platform as part of the protocol and the fees is calculated using the on-chain oracles.

Upon checking the implementation of both the functions: stake() & withdraw(), there is a strict issue that tends to fail causing the users to revert.

```
function stake(IERC721 _collection, uint256[] calldata _tokenIds) external
payable rewardRedemption(false) nonReentrant{
    ...
    ...
    uint256 totalStakingFee = getStakeFee();
    if(msg.value != totalStakingFee){
        revert InsufficientStakingFee();
    }
    ...
    ...
}
```

Assume, the `totalStakingFee` is 0.3 Ether and the user has decided to pay around 0.5 ether (including gas-fee for the faster transaction) and now, as the `msg.value` is > 0.3 ether, the function always fails even though the user has paid more amount leading to revert thus making him not to stake/withdraw his tokens.

Impact

The function always reverts if the user has provided more native-token as fee for various reasons and not being able to validate those leads to the issue of not being interactive with the protocol.

Recommendation

Change the functionality to: [Where you're also sending back the tokens if excess]

```
require(msg.value >= totalStakingFee);
uint256 RefundAmount = msg.value - totalStakingFee;
address(payable(msg.sender)).call{msg.value = RefundAmount}("");
```

Status

Resolved

[H-05] KingdomlyStaking#withdrawRewardToken - Incorrect Handling of depositedRewardTokenAmount variable leading to potential issues

Description

The `withdrawRewardToken()` function in the `KingdomlyNFTStakingPool` contract does not reset the `depositedRewardTokenAmount` to zero after transferring reward tokens to the owner. This oversight can lead to incorrect accounting of available reward tokens, causing potential over withdrawal attempts and transaction reverts due to insufficient balance.

Vulnerability Details

Consider the following sequence of actions:

- Initial State:

→ `depositedRewardTokenAmount` = 1000 tokens

→ Actual balance of reward tokens in the contract = 1000 tokens

- First Withdrawal:

→ The owner calls `withdrawRewardToken()`.

→ All 1000 tokens are transferred to the owner.

→ `depositedRewardTokenAmount` remains 1000 (not reset to 0).

→ Actual balance of reward tokens in the contract = 0 tokens

- Subsequent Deposit:

→ The owner deposits 500 more tokens using `depositRewardToken()` or `increaseDepositedRewardTokenAmount()`.

→ `depositedRewardTokenAmount` is now 1500 tokens.

→ Actual balance of reward tokens in the contract = 500 tokens

- Second Withdrawal:

→ The owner calls `withdrawRewardToken()` again.

→ The function attempts to withdraw 1500 tokens (the value of `depositedRewardTokenAmount`).

However, only 500 tokens are actually in the contract, causing the transaction to revert due to insufficient balance.

Impact

- **Incorrect Reward Accounting:** Any function relying on `depositedRewardTokenAmount` as a source of truth for available rewards will have incorrect information.
- **Inflated Reward Display:** Users might see inflated reward amounts if the UI uses `depositedRewardTokenAmount`.
- **Failed Transactions:** Reward calculations and distributions based on `depositedRewardTokenAmount` will be incorrect, leading to failed transactions.

Recommendation

To fix this issue, modify the `withdrawRewardToken` function to reset `depositedRewardTokenAmount` to zero after the transfer:

```
function withdrawRewardToken()external onlyOwner rewardRedemption(false){
    if(address(rewardToken) == address(0)){
        revert InvalidTokenAddress();
    }
    uint256 amountToWithdraw = rewardToken.balanceOf(address(this));
    depositedRewardTokenAmount = 0;
    rewardToken.safeTransfer(msg.sender,amountToWithdraw);
    emit RewardTokenWithdrawn(address(rewardToken), amountToWithdraw);
}
```

Status

Resolved

Medium

[M-01]

KingdomlyStaking#calculateUserPointsFromStakedTokens/calculateUserCurrentlyStakedTokenPoints - Precision Loss in StakedDays Calculation

Description

The calculation for determining the number of days a token has been staked in the KingdomlyNFTStakingPool contract uses integer division, which results in precision loss. The formula $(\text{calculationTimestamp} - \text{userStakedTokens}[i].\text{stakedAt}) / 86400$ rounds down to the nearest whole day, potentially underestimating the staking duration. This can lead to users receiving fewer points than they should for partial days of staking.

Vulnerability Details

Staking and Calculation:

→ A user stakes a token at timestamp 1,000,000.

→ The calculationTimestamp is 1,085,999, which is 1 day and 23 hours later.

Current Calculation:

The calculation $(1,085,999 - 1,000,000) / 86400 = 85,999 / 86,400 = 0$ days.

→ Despite nearly 2 days of staking, the user receives points for 0 days.

Result:

Users receive fewer rewards than they should for partial days of staking.

The total reward distribution becomes skewed, disproportionately favoring longer-term stakers.

Impact

Users receive fewer rewards than they should for partial days of staking.

The total reward distribution becomes skewed, favouring longer-term stakers disproportionately.

It creates an unfair advantage for users who can time their stakes to align perfectly with daily boundaries.

Recommendation

To address this issue, the contract could:

Use a Smaller Time Unit: Consider using hours or minutes instead of days for a more precise calculation.

Implement Precise Calculation: Account for fractional days in the calculation.

Status

Resolved

[M-02] KingdomlyFeeContract - Single Point of failure in Oracle Dependency

Description

The KingdomlyFeeContract relies solely on the Pyth oracle to fetch the Ethereum price for calculating the equivalent of one dollar in Wei. This creates a single point of failure, as any compromise or downtime in the Pyth oracle would render the contract's critical functions unreliable. This dependency poses a significant risk to the contract's stability and the accuracy of its fee calculations.

Oracle Downtime:

The Pyth oracle experiences downtime or is compromised.

The `getOneDollarInWei` and `updateOracleAndGetOneDollarInWei` functions become unable to fetch the necessary price data.

Impact

Users cannot accurately calculate or update fees, potentially leading to transaction failures.

The contract's operations are disrupted, affecting all users and dependent contracts.

Recommendation

To address this issue, consider implementing the following strategies:

- Redundancy: Integrate multiple oracle providers to ensure price data is available even if one oracle fails.

- Fallback Mechanism: Implement a fallback mechanism that uses a cached price or a different data source if the primary oracle is unavailable.
- Monitoring and Alerts: Set up monitoring and alert systems to detect oracle failures and take corrective actions promptly.

Status

Resolved

[M-03] KingdomlyStaking#withdrawRewardToken - Lack of checks for ActiveStakes & PendingRewards leads to not being able to claim rewards for users

Description

The `withdrawRewardToken` function in the `KingdomlyNFTStakingPool` contract allows the owner to withdraw all deposited reward tokens without checking for active stakes or pending rewards. This oversight can lead to situations where users who have earned rewards are unable to claim them, as the tokens may have already been withdrawn by the owner

Vulnerability Details

Consider the following scenario:

- Initial State:

→ Total reward tokens in the contract: 1000 tokens
→ User A has staked tokens and earned 100 reward tokens
→ User B has staked tokens and earned 50 reward tokens
→ Remaining 850 tokens are for future rewards

- Owner Withdrawal:

→ The owner calls `withdrawRewardToken()`.
→ All 1000 tokens are transferred to the owner.

No check is performed for active stakes or pending rewards.

- Consequences:

→ User A tries to claim their 100 earned tokens, but the transaction fails because there are no tokens left in the contract.

→ User B also cannot claim their 50 earned tokens.

Any newly accruing rewards for current stakers cannot be distributed.

The staking mechanism breaks down as there are no rewards to distribute.

Impact

If the contract does not have a way to pause staking, users might continue to stake without receiving rewards.

The contract's accounting of owed rewards becomes incorrect, as it still shows users are owed rewards that no longer exist in the contract.

Recommendation

To mitigate this issue, the `withdrawRewardToken` function should include checks for active stakes and pending rewards. Implement a mechanism to calculate total pending rewards and ensure only tokens not owed to current stakers can be withdrawn:

```
function withdrawRewardToken()external onlyOwner rewardRedemption(false){
    ...
    ...
    // New function to calculate Pending Rewards

    uint256 totalPendingRewards = calculateTotalPendingRewards();
    uint256 availableForWithdrawl  =  token.balanceOf(address(this))  -
totalPendingRewards;

    ...
    ...
}
```

Status

Resolved

[M-04] KingdomlyStaking#startRewardRedemption_OFF_CHAIN - Lack of Input Validation

Description

The `startRewardRedemption_OFF_CHAIN` function in the `KingdomlyNFTStakingPool` contract allows an external input (`_totalPoolPoints`) to directly modify the `totalPoolPoints` state variable without any validation. This lack of input validation poses a significant security risk, as it could be exploited to manipulate the reward distribution system, leading to potential financial losses and unfair distribution of rewards.

Vulnerability Details

- The contract has 1,000,000 tokens staked by various users.
- The actual total pool points, based on staking duration and amounts, should be 5,000,000.
- A malicious owner or compromised off-chain calculation system provides an inflated value:
 - `contract.startRewardRedemption_OFF_CHAIN(1000000000000); // Extremely inflated value`
 - The `totalPoolPoints` is set to this inflated value.
 - When users try to redeem rewards, their share of the total pool is calculated as:
 - $\text{userRewardPercentage} = (\text{userPoints} * 1_000_000) / \text{totalPoolPoints}$
 - Due to the inflated `totalPoolPoints`, legitimate users receive a tiny fraction of the rewards they should have received. For example, a user who should receive 10% of the rewards might now receive:
 - $(500000 * 1_000_000) / 1000000000000 = 0.0005\%$ of the rewards
 - This effectively steals rewards from legitimate users and allows the attacker to claim an outsized portion of the rewards.

Recommendation

To mitigate this issue, the following measures should be implemented:

Input Validation: Implement strict validation checks on the `_totalPoolPoints` input to ensure it is within a reasonable range based on known parameters like total staked tokens and maximum staking duration.

Multi-Signature or Time-Lock Mechanism: Consider using a multi-signature or time-lock mechanism for critical functions to prevent unilateral actions by a single party.

Maximum Limit: Set a maximum limit for `totalPoolPoints` to prevent unrealistic values from being set.

Event Logging: Add events to log the old and new `totalPoolPoints` for transparency and auditing purposes.

Status

Acknowledged

[M-05]

KingdomlyStaking#depositRewardToken&increaseDepositedRewardTokenAmount - Inconsistent Reward Token Deposit Mechanism and Potential Loss of Deposited Rewards

Description

1. The contract implements two separate functions for depositing reward tokens: `depositRewardToken` and `increaseDepositedRewardTokenAmount`. This redundancy complicates the contract and increases the potential for errors.
2. The `depositRewardToken` function lacks a check to prevent multiple calls, which can lead to unintended consequences:
 - a. If called with a different token address, it overwrites the previously set reward token without any way to revert the change.
 - b. If called with a different amount, it overwrites the `depositedRewardTokenAmount`, potentially losing track of previously deposited rewards.
3. In a scenario where `depositRewardToken` is called after previous deposits (including those made through `increaseDepositedRewardTokenAmount`), all previous deposits are effectively lost as the `depositedRewardTokenAmount` is overwritten.

Recommendation

1. Merge `depositRewardToken` and `increaseDepositedRewardTokenAmount` into a single function that handles both initial deposits and increases:

```
Function depositRewardToken(IERC20 _token, uint256 _amount) external onlyOwner
rewardRedemptionActive(false) {

    if (_amount == 0) {

        revert InvalidRewardTokenAmount();

    }

    if (address(rewardToken) == address(0)) {

        rewardToken = _token;

    } else if (address(_token) != address(rewardToken)) {

        revert InvalidTokenAddress();

    }

    depositedRewardTokenAmount += _amount;

    rewardToken.safeTransferFrom(msg.sender, address(this), _amount);

    emit RewardTokenDeposited(address(_token), _amount);

}
```

2. Implement a function to allow changing the reward token, but only if no deposits have been made:

```

Function      changeRewardToken(IERC20      _newToken)      external      onlyOwner
rewardRedemptionActive(false) {

    if (depositedRewardTokenAmount > 0) {

        revert RewardTokenAlreadyDeposited();

    }

    rewardToken = _newToken;

    emit RewardTokenChanged(address(_newToken));

}

```

Status

Resolved

Low

[L-01] KingdomlyNFTStakingPool#constructor - Duplicate NFT Collections in Constructor Leading to Data Overwrite

Description

In the KingdomlyNFTStakingPool contract, the constructor processes an array of NFT collections and their corresponding daily points. If the `_collections` array contains duplicate addresses, the state variables related to these collections are overwritten. This results in incorrect data being stored, particularly affecting the daily points assigned to each collection. The contract assumes that the `_collections` array contains unique addresses, which is not enforced, leading to potential data integrity issues.

Vulnerability Details

Consider the following input arrays passed to the constructor:


```
IERC721[] memory _collections = new IERC721[](3);
_collections[0] = IERC721(0x123...); // Collection A
_collections[1] = IERC721(0x123...); // Collection A (duplicate)
_collections[2] = IERC721(0x456...); // Collection B
```

```
uint256[] memory _dailyPoints = new uint256[](3);
_dailyPoints[0] = 100; // Points for Collection A
_dailyPoints[1] = 200; // Points for duplicate Collection A
_dailyPoints[2] = 150; // Points for Collection B
```

During constructor execution, the following issues occur:

collectionsIndex: The index for 0x123... is overwritten to point to the last occurrence.

collectionDailyPoints: The daily points for 0x123... are set to 200, overwriting the initial 100 points.

collectionSupported: Remains true for 0x123... but does not differentiate between the duplicates.

Impact

- **Overwritten Values:** The daily points for duplicate collections are set to the value of the last occurrence, leading to incorrect data.
- **Loss of Data:** The initial daily points for duplicate collections are lost, compromising data integrity.

Recommendation

To prevent these issues, the constructor should enforce uniqueness in the `_collections` array. This can be achieved by adding a logic check, such as a `require` statement, to validate that each address in `_collections` is unique before processing. This ensures that each collection is stored correctly in the associated mappings, preserving their respective daily points and support statuses.

Status

Resolved

[L-02] KingdomlyStaking#Constructor - Inadequate Input Validations for Admin and Fee Contract Addresses

Description

The KingdomlyNFTStakingPool contract's constructor lacks validation for the `_kingdomlyAdmin` and `_kingdomlyFeeContract` parameters, allowing them to be set to zero addresses. This oversight can lead to significant operational issues, including contract lockout and failed fee calculations, as these addresses are critical for administrative functions and fee operations.

Vulnerability Details

Consider the following contract deployment scenario:

```
KingdomlyNFTStakingPool pool = new KingdomlyNFTStakingPool(  
    "My Staking Pool",  
    new IERC721[](2), // Valid array of ERC721 collections  
    new uint256[](2), // Corresponding daily points  
    address(0), // Zero address for kingdomlyAdmin  
    KingdomlyFeeContract(0) // Zero address for kingdomlyFeeContract  
);
```

During execution, the constructor does not validate whether `_kingdomlyAdmin` and `_kingdomlyFeeContract` are zero addresses, resulting in:

- `kingdomlyAdmin` set to `address(0)`.
- `kingdomlyFeeContract` set to `address(0)`.

Impact

- **Authority Issues:** Functions restricted to the admin cannot be executed, as `address(0)` lacks permissions, potentially locking out critical administrative functions.
- **Fee Calculations:** Operations requiring the `kingdomlyFeeContract` will fail, as they attempt to interact with a non-existent contract, leading to transaction failures and incorrect fee processing.

Recommendation

To prevent these issues, the constructor should include validation checks to ensure that both `_kingdomlyAdmin` and `_kingdomlyFeeContract` are valid, non-zero addresses. Implement the following checks:

```
require(_kingdomlyAdmin != address(0), "Kingdomly Admin cannot be Zero.");  
require(address(_kingdomlyFeeContract) != address(0), "KingdomlyFee contract  
cannot be zero");
```

Status

Resolved

[L-03] KingdomlyStaking#redeemRewards - Incorrect Event Parameter in RewardPointsRedeemed Event

Description

The `redeemRewards` function in the `KingdomlyNFTStakingPool` contract emits a `RewardPointsRedeemed` event with an incorrect parameter. Instead of emitting the `totalUserPoints`, which represents the user's total redeemed points, the event incorrectly emits `depositedRewardTokenAmount`. This discrepancy can lead to inaccurate off-chain tracking of redeemed points, affecting analytics and user trust.

Vulnerability Details & Impact

- **Function Execution:** A user calls the `redeemRewards` function to redeem their accumulated reward points. The function calculates the user's total points (`totalUserPoints`) and sets it to the user's redeemed points.
- **Event Emission:** The `RewardPointsRedeemed` event is emitted with `depositedRewardTokenAmount` instead of `totalUserPoints`.
- **Result:** Off-chain systems relying on the event for tracking redeemed points will record incorrect data, as they receive `depositedRewardTokenAmount` instead of the actual points redeemed by the user.

Recommendation

To resolve this issue, the event parameter should be corrected to emit the actual `totalUserPoints`:

```
emit RewardPointsRedeemed(msg.sender, totalUserPoints);
```

Status

Resolved

[L-04] KingdomlyFeeContract#getOneDollarInWei - Present Pyth-Oracle representation works only on Networks with native-Token of Decimals 18

Description

The `getOneDollarInWei()` function in the `KingdomlyFeeContract` assumes that the native token on any blockchain network has 18 decimals. This assumption fails on networks where the native token has different decimals, such as 9 decimals on SaitaChain. As a result, the function may calculate incorrect values for \$1 in wei, leading to potential discrepancies.

Vulnerability Details

The `getOneDollarInWei()` function in the `KingdomlyFeeContract` assumes that the native token on any blockchain network has 18 decimals. This assumption fails on networks where the native token has different decimals, such as 9 decimals on SaitaChain. As a result, the function may calculate incorrect values for \$1 in wei, leading to potential discrepancies.

Impact

The incorrect calculation of \$1 in wei due to varying native token decimals across different blockchain networks could cause significant issues, such as overcharging or undercharging users, especially if the protocol is deployed on multiple chains. This could

lead to user dissatisfaction and financial discrepancies.

Recommendation

Change the underlying code in `getOneDollarInWei()` function, depending upon the native-token decimal value to effectively get the value of \$1.

Status

Acknowledged

[L-05] KingdomlyStaking#startRewardRedemption - Discrepancy in Reward Points Calculation

Description

The `startRewardRedemption` function in the `KingdomlyNFTStakingPool` contract is documented to calculate the total points from staked tokens on-chain when the reward redemption process starts. However, the actual implementation of the function does not perform this calculation, leading to potential discrepancies in reward distribution. This could cause issues if other parts of the system rely on the assumption that total points are calculated at this stage.

Proof of Concept (PoC):

Consider the following scenario where this discrepancy could cause issues:

The contract has a separate function, `distributeRewards`, which distributes rewards based on total points:

```
function distributeRewards() external {  
  
    require(rewardRedemptionStarted, "Redemption not started");  
  
    uint256 totalPoints = getTotalPoints(); // Assumes total points were calculated  
  
    // ... distribution logic ...  
  
}
```

A developer assumes that total points are calculated when redemption starts, based on the comment in startRewardRedemption:

```
function getTotalPoints() internal view returns (uint256) {  
    require(rewardRedemptionStarted, "Redemption not started");  
    return totalPoolPoints; // Assumes this was set in startRewardRedemption()  
}
```

The contract owner calls startRewardRedemption:

```
owner.call(contract.startRewardRedemption());
```

Later, when distributeRewards is called, it uses getTotalPoints, which returns 0 or an outdated value, leading to incorrect reward distribution.

Recommendation

To resolve this issue, the function should either perform the calculation as described in the comment, or the comment should be updated to accurately reflect the function's actual behaviour. If the calculation of total points is necessary, it should be implemented correctly and called at the appropriate time, ensuring that the total pool points are accurately set before any reward distribution logic relies on them.

Status

Resolved

Gas

[G-01]

KingdomlyStaking/calculateUserPointsFromStakedTokens&calculateUserCurrentlyStakedTokenPoints - Superfluous Initialization of Variables In Functions

Description

In the KingdomlyStaking.sol contract, the variable userCalculatedRewards is explicitly initialised to zero in two functions: calculateUserCurrentlyStakedTokenPoints and calculateUserPointsFromStakedTokens. Since Solidity automatically initialises variables to their default values, including zero for numeric types, this explicit initialization is unnecessary and can be removed to simplify the code.

Location:

-> KingdomlyStaking.sol at line 607

-> KingdomlyStaking.sol at line 648

Example:

Current code:

```
uint256 userCalculatedRewards = 0;
```

This line appears in both the calculateUserCurrentlyStakedTokenPoints and calculateUserPointsFromStakedTokens functions, where userCalculatedRewards is used to accumulate reward points.

Recommendation

Remove the explicit initialization of userCalculatedRewards to zero, as it is redundant. The updated code should simply declare the variable without initialization:

```
uint256 userCalculatedRewards;
```

Status

Resolved

Centralisation

The Kingdomly Project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised



Conclusion

After Hashlocks analysis, the Kingdomly Project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au

#Hashlock.



#Hashlock.

Hashlock Pty Ltd